SECTION 1

Look at the Big Picture

- **Business objective:** Predicted medians feed a downstream investment pipeline, so framing the problem starts with how errors affect revenue, not only with algorithms.
- **Problem type:** The task is supervised, univariate regression with multiple input features, using batch learning on data that fits in memory.
- **Performance metric:** RMSE (and related squared-error views) emphasize large prediction mistakes, which matches costly pricing errors better than mean absolute error in many business settings.
- **Assumptions:** Labels must align with how predictions will be consumed; verifying downstream use avoids optimizing the wrong target.

SECTION 2

Get the Data

- **Workspace and tooling:** A repeatable environment with Jupyter, NumPy, pandas, Matplotlib, and Scikit-Learn supports exploration and training scripts.
- **Ingestion:** Housing data is loaded into pandas DataFrames; `info()` and `describe()` reveal dtypes, missing counts, and distribution tails.
- **Test set discipline:** Hold out test data before deep exploration so insights do not leak into evaluation.
- **Stable splitting:** Hash-based or identifier-aware splits plus `train_test_split` with `random_state` keep test sets consistent across runs and dataset updates.

SECTION 3

Discover and Visualize the Data to Gain Insights

- **Geography:** Scatter plots of latitude and longitude show coastal clusters and dense urban corridors tied to price patterns.
- **Correlations:** Pearson *r* highlights median income as the strongest linear correlate; scatter matrices expose nonlinear caps and plateaus in prices.
- **Sampling bias lesson:** Stratified sampling on binned income keeps train and test representative of rare high-income districts, unlike naive random splits.
- **Attribute combinations:** Engineered ratios such as rooms per household and bedrooms per room can correlate more strongly with price than raw totals.

SECTION 4

Prepare the Data for Machine Learning Algorithms

- **Cleaning:** Missing numeric values are imputed with medians fit on training data; statistics must be reused at scoring time.
- **Categories:** `ocean_proximity` is encoded with one-hot vectors rather than arbitrary ordinals that imply false order.
- **Custom transformers:** Classes mixing in `BaseEstimator` and `TransformerMixin` implement `fit` / `transform` so custom feature logic plugs into tuning.
- **Scaling and pipelines:** `StandardScaler` (or min-max) fits on train only; `Pipeline` and `ColumnTransformer` chain imputation, custom steps, scaling, and encoding without leakage.

SECTION 5

Select and Train a Model

- **Baselines:** A quick `LinearRegression` fit establishes a simple reference on prepared features.
- **Overfitting trap:** A `DecisionTreeRegressor` can hit zero training error while cross-validated RMSE is worse than linear models—memorization is not generalization.
- **Cross-validation:** `cross_val_score` with *k* folds yields mean and spread of RMSE, using negative MSE scoring conventions in Scikit-Learn.
- **Ensembles:** `RandomForestRegressor` often lowers validation error but may still overfit training folds, motivating regularization or more data.

SECTION 6

Fine-Tune Your Model

- **Grid search:** `GridSearchCV` cross-validates every listed hyperparameter combination and can refit the best estimator on all training data.
- **Randomized search:** `RandomizedSearchCV` explores large spaces by sampling random hyperparameter draws under a fixed budget.
- **Pipelines in search:** Preprocessing steps exposed as hyperparameters let search decide whether engineered attributes help.
- **Error analysis:** Feature importances and inspection of large residuals guide dropping weak inputs or fixing quirks like price caps.

SECTION 7

Launch, Monitor, and Maintain Your System

- **Test-set evaluation:** The chosen pipeline `transform`s the test set (no refit) for a final RMSE estimate, optionally with a confidence interval on squared errors.
- **Production:** Models ship via `joblib` inside the full preprocessing pipeline, often behind a web service or cloud endpoint that loads once at startup.
- **Monitoring:** Live metrics—directly or via downstream KPIs—detect drift and broken pipelines; alerts should precede silent quality decay.
- **Retraining:** Evolving distributions require fresh data and periodic retraining, sometimes automated in batch pipelines.